

Client–Server Communication in Python using TCP, UDP, and Video Streaming

Sharique Ahmad

14-01-2024

Project 1: Socket Programming using TCP and UDP

Overview

The project demonstrates client–server communication using both TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

- In the TCP version, the client can send either text or a file:
 - For text: the server receives the message, converts it to uppercase, and sends it back.
 - For file: the server receives the file in binary form and saves it with the prefix `received_`.
- In the UDP version, the client sends a line of text and the server replies with the same text converted to uppercase.

This demonstrates the difference between connection–oriented (TCP) and connection–less (UDP) communication.

TCP Implementation

- The server creates a TCP socket, binds it to the address: `server_address = ('localhost', 12345)` and listens for incoming connections.
- When a client connects, it can send either a text message or a file, as described in Section 1.1.
- The client must use the same address and port to connect.

UDP Implementation

- The server creates a UDP socket bound to the address: `server_address = ('localhost', 54321)` and waits for datagrams.
- On receiving a message, it converts the text to uppercase and sends it back.
- The client must use the same address and port to send datagrams and then display the reply.

How to Run

1. Open two terminals.
2. In the first terminal, start the server:

```
python server_TCP.py  
or  
python server_UDP.py
```

3. In the second terminal, run the client:

```
python client_TCP.py  
or  
python client_UDP.py
```

4. For TCP:

- If the client sends text, the server will return the text in uppercase.
- If the client sends a file, the server saves it with the prefix `received_`.

For UDP:

- Enter a line of text. The server will return the text in uppercase.

Working Principle

- In TCP, a reliable connection is established, ensuring ordered delivery of data.
- In UDP, messages are sent as datagrams without a connection, making it faster but less reliable.
- Both implementations follow the same basic socket workflow:
 1. Create socket
 2. Bind/Connect
 3. Send/Receive
 4. Close connection

Project 2: Real-Time Video Streaming using TCP

Overview

This project demonstrates real-time video streaming using TCP socket programming in Python. The server captures frames from a webcam using OpenCV, serializes them, and transmits them to a client. The client deserializes and displays the video in real time.

Server Implementation

- Creates a TCP socket bound to the address: ('localhost', 5555).
- Accepts an incoming client connection.
- Captures video frames from the webcam.
- Serializes the frames using `pickle`.
- Sends the frame size (using `struct.pack`) followed by the frame data.

Client Implementation

- Connects to the server at ('localhost', 5555).
- Receives the size of each incoming frame.
- Receives the frame data and deserializes it using `pickle`.
- Displays the video frames in a window using OpenCV.
- The stream ends when the user presses `q`.

How to Run

1. Open two terminals.
2. In the first terminal, start the server:

```
python Server.py
```

3. In the second terminal, run the client:

```
python client.py
```

4. The client window will display the video.
5. Press `q` to stop streaming.

Working Principle

- The server continuously reads webcam frames and sends them across the TCP connection.
- Each frame is preceded by its size so the client knows how many bytes to read.
- The client reconstructs and displays frames in real time.
- TCP ensures ordered and reliable transmission, which is crucial for video.